

P2249R6

Mixed comparisons for smart pointers

Published Proposal, 2024-02-02

Author:

[Giuseppe D'Angelo](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose to enable mixed comparisons for the Standard Library smart pointer class templates `unique_ptr` and `shared_ptr`, so that one can compare them against raw pointers.

Table of Contents

1	Changelog
2	Tony Tables
3	Motivation and Scope
3.1	Associative containers
4	Impact On The Standard
5	Design Decisions
5.1	What is the signature of the proposed operators?
5.1.1	Allowing mixed comparisons against raw pointers
5.1.2	Handling of array types
5.1.3	Custom pointer types for <code>unique_ptr</code>
5.1.4	Objects comparable to a smart pointer's pointer type
5.1.5	Should mixed comparisons between <i>different</i> Standard Library smart pointer classes be allowed?
5.2	Should the comparison operators yield a total order?
5.3	Would these operations make the <code>equality_comparable_with</code> or <code>three_way_comparable_with</code> concepts satisfied between a smart pointer and a raw pointer?
5.4	Should <code>unique_ptr</code> have the full set of ordering operators (<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>), or just <code><=></code> ?
6	Technical Specifications
6.1	Feature testing macro #
6.2	Proposed wording
6.2.1	<code>unique_ptr</code>
6.2.2	<code>shared_ptr</code>
7	Implementation experience
8	Acknowledgements
	References

§ 1. Changelog

- R6
 - Reduce the scope of the proposal: constrain the proposed operators to work only with raw pointers, to avoid clashes with user-defined overloads.
- R5
 - Redesigned the overload set.
 - Made the `unique_ptr` operators `constexpr`, following the adoption of [\[P2273R3\]](#).
- R4
 - Corrected the references to [\[P2405R0\]](#) (which had been accidentally referred to as P2403).
- R3
 - Clarified some design decisions after a review on the LEWG mailing list.
 - Changed the proposed wording for the constraints, in order to exclude types that are derived from `unique_ptr` or `shared_ptr` specializations.
- R2
 - Fixes to the proposed wording.
- R1
 - Added some clarifications to the design decisions, following a review on the LEWG mailing list.
 - Made the proposed operators hidden friends.
 - Rebased on top of the latest draft of the Standard.
 - Minor fixes.
- R0
 - First submission.

§ 2. Tony Tables

Before

```
class Manager
{
    // The Manager owns the Objects, therefore it uses smart pointers.
    // The Manager gives out non-owning raw pointers to clients.
    // A client gives the raw pointer back to the Manager, to tell it
    // to act on that Object; the Manager has then to look up the Object
    // in its storage.

    std::vector<std::unique_ptr<Object>> objects;

public:
    Object* get_object(~~~) const
    {
        return objects[~~~].get();
    }

    void drop_object(Object *input)
    {
        // Must use erase_if and a custom comparison (e.g. a lambda).
        auto isEqual = [input](const std::unique_ptr<Object> &o) {
            return o.get() == input;
        };
        erase_if(objects, input);
    }

    ssize_t index_for_object(Object *input) const
    {
        // Same story.
        // Code like this (predicates, etc.) may get duplicated all over the place
        // where smart pointers are used in containers/algorithms. Surely,
        // centralizing it is good practice, but there's always the temptation of
        // just writing the one-liner lambda and "moving on" rather than refactoring...
        auto isEqual = [input](const std::unique_ptr<Object> &o) {
            return o.get() == input;
        };
        auto it = std::ranges::find_if(objects, isEqual);
        // etc.
    }
};
```

```
class Manager
{
    std::vec

public:
    Object *
    {
        retu
    }

    void dro
    {

        // J
        eras

    }

    ssize_t
    {

        // S
        // U
        // s
        // i
        auto
        // e

    }
};
```

```
// Suppose instead that the Manager needs to use an associative container rather than
// a sequential container (e.g. mapping some data to each object).
// Then, an heterogeneous comparator becomes a necessity -- we can't possibly
// look up a unique_ptr using another unique_ptr to the same object, especially
// if clients give us non-owning raw pointers to act upon.
```

```
// Heterogeneous comparator
template <class T> struct smart_pointer_comparator {
    struct is_transparent {};

    bool operator()(const std::unique_ptr<T> &lhs, const std::unique_ptr<T> &rhs) const
    { return lhs < rhs; }
    bool operator()(const std::unique_ptr<T> &lhs, const T *rhs) const
    { return std::less()(lhs.get(), rhs); }
    bool operator()(const T *lhs, const std::unique_ptr<T> &rhs) const
    { return std::less()(lhs, rhs.get()); }
};
```

```
// A sorted associative container with some data
std::map<std::unique_ptr<Object>, Data,
```

```
// No need f
```

```
// ... just
std::map<std
```

Before	
<pre> smart_pointer_comparator<Object>> objects = ~~~; // Heterogeneous lookup using a raw pointer object *ptr = ~~~; auto it = objects.find(ptr); if (it != objects.end()) { use(it->second); } </pre>	<pre> std::less<Object*> ptr auto it = ob if (it != ob </pre>
<pre> // Same, with an unordered associative container. // Heterogeneous hasher; [util.smartptr.hash] guarantees that both the // specializations below return the very same value for the same pointer. template <class T> struct smart_pointer_hasher { struct is_transparent {}; size_t operator()(const std::unique_ptr<T>& ptr) const { // equal by definition to std::hash<T*>(ptr.get()), that is, (*this)(ptr.get()) return std::hash<std::unique_ptr<T>>()(ptr); } size_t operator()(T* ptr) const { return std::hash<T*>()(ptr); } }; // Heterogeneous equality comparator template <class T> struct smart_pointer_equal { struct is_transparent {}; bool operator()(const std::unique_ptr<T>& lhs, const std::unique_ptr<T>& rhs) const { return lhs == rhs; } bool operator()(const std::unique_ptr<T>& lhs, const T* rhs) const { return lhs.get() == rhs; } bool operator()(const T* lhs, const std::unique_ptr<T>& rhs) const { return lhs == rhs.get(); } }; std::unordered_map<std::unique_ptr<Object>, Data, smart_pointer_hasher<Object>, smart_pointer_equal<Object>> objects = ~~~; // Heterogeneous lookup Object* ptr = ~~~; auto it = objects.find(ptr); if (it != objects.end()) { use(it->second); } </pre>	<pre> // [P0919R3] // so a cust template <cl { struct i size_t c // e retu } size_t c retu } }; // Custom he std::unordered smart_pc std::equ // Heterogen Object* ptr auto it = ob if (it != ob </pre>

§ 3. Motivation and Scope

Smart pointer classes are universally recognized as the idiomatic way to express ownership of a resource (very incomplete list: [Sutter], [Meyers], [R.20]). On the other hand, raw pointers (and references) are supposed to be used as non-owning types to access a resource.

Both smart pointers and raw pointers, as their name says, share a common semantic: **representing the address of an object**.

This semantic comes with a set of meaningful operations; for instance, asking if two (smart) pointers represent the address of the same object. `operator==` is used to express this intent.

Indeed, with the owning smart pointer class templates available in the Standard Library (`unique_ptr` and `shared_ptr`), one can already use `operator==` between two smart pointer objects (of the same class). However one *cannot* use it

between a smart pointer and a raw pointer, because the Standard Library is lacking that set of overloads; instead, one has to manually extract the raw pointer out of the smart pointer class:

```
std::shared_ptr<object> sptr1, sptr2;
object *rawptr;

// Do both pointers refer to the same object?
if (sptr1 == sptr2) { ~~~ } // WORKS
if (sptr1 == rawptr) { ~~~ } // ERROR, no such operator
if (sptr1.get() == rawptr) { ~~~ } // WORKS; but why the extra syntax?
```

This discussion can be easily generalized to the full set of the six relational operations; these operations have already well-established semantics, and are indeed already defined between smart pointers objects (of the same class) or between raw pointers, but they are not supported in mixed scenarios.

We propose to remove this inconsistency by defining the relational operators between the Standard Library owning smart pointer classes and raw pointers.

Allowing mixed comparisons isn't merely a "semantic fixup"; the situation where one has to compare smart pointers and raw pointers commonly occurs in practice (the typical use case is outlined in the first example in the [§ 2 Tony Tables](#) above, where a "manager" object gives non-owning raw pointers to clients, and the clients pass these raw pointers back to the manager, and now the manager needs to do mixed comparisons).

§ 3.1. Associative containers

Moreover, we believe that allowing mixed comparisons is useful in order to streamline heterogeneous comparison in associative containers for smart pointer classes.

The case of an associative container using a `unique_ptr` as its key type is particularly annoying; one cannot practically ever look up in such a container using another `unique_ptr`, as that would imply having two `unique_ptr` objects owning the same object. Instead, the typical lookup is heterogeneous (by raw pointer); this proposal is one step towards making it more convenient to use, because it enables the usage of the standard `std::less` or `std::equal_to`.

We however are not addressing at all the issue of heterogeneous hashing for smart pointers. While likely very useful in general, heterogeneous hashing can be tackled separately by another proposal that builds on top of this one, for instance, by making the `std::hash` specializations for Standard smart pointers 1) *transparent*, and 2) able to hash the smart pointer's `pointer_type` / `element_type*` as well as the smart pointer object itself. More research and field experience is needed.

§ 4. Impact On The Standard

This proposal is a pure library extension. It proposes changes to an existing header, `<memory>`, but it does not require changes to any standard classes or functions and it does not require changes to any of the standard requirement tables. The impact is positive: code that was ill-formed before becomes well-formed.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

[\[P0805R2\]](#) is vaguely related to this proposal. It proposes to add mixed comparisons between containers of the same type (for instance, to be able to compare a `vector<int>` with a `vector<long>`), without resorting to manual calls to algorithms; instead, one can use a comparison operator. A quite verbose call to `std::equal(v1.cbegin(), v1.cend(), v2.cbegin(), v2.cend())` can therefore be replaced by a much simpler `v1 == v2`. In this sense, [\[P0805R2\]](#) matches the spirit of the current proposal, although comparing smart pointers and raw pointer does not require any algorithm, and does not have such a verbose syntax.

§ 5. Design Decisions

§ 5.1. What is the signature of the proposed operators?

The motivating reason of this proposal is that this code should work:

```
smart_pointer<T> smartptr = ~~~;
T *rawptr = ~~~;

if (smartptr == rawptr) { ~~~ } // OK with this proposal
```

We could therefore conclude that for instance `operator==` should have this signature:

```
// not proposed
template <typename T>
constexpr bool operator==(const smart_pointer<T> &lhs, const T *rhs) noexcept;
```

(The other operators would have a similar one.)

While this signature "make sense", there is however a number of subtleties that need to be taken into account and are going to be addressed in the next sections.

§ 5.1.1. Allowing mixed comparisons against raw pointers

The rationale of this proposal is that smart pointers should act like raw pointers when it comes to comparison operators. For instance, raw pointers allow for mixed comparisons if both pointers are convertible to their composite pointer type ([`expr.type`]):

```
Base *b = ~~~;
Derived *d = ~~~;

if (b == d) { ~~~ } // OK
```

Therefore, we want the following to also work:

```
smart_pointer<Base> b = ~~~;
Derived *d = ~~~;

if (b == d) { ~~~ } // Should also be OK with this proposal
```

In this case a signature like `operator==(const smart_pointer<T> &lhs, const T *rhs)` would not compile, because we cannot deduce `T` in the call above (is it `Base` or is it `Derived`?). For this reason, we need to generalize the signature of the comparison operators, and add constraints to it:

```
// not proposed
template <typename T, typename U>
    requires equality_comparable<T *, U *>
constexpr bool operator==(const smart_pointer<T> &lhs, const U *rhs) noexcept;
```

§ 5.1.2. Handling of array types

The Standard Library smart pointers can be instantiated with array types. This requires some special handling in the operator signatures: for a smart pointer `smart_pointer<T>`, we can't just check if `T *` is comparable with `U *`, because `T` may be an array type.

Therefore, we have to amend the constraint, and use:

- `typename std::unique_ptr<T>::pointer` for `std::unique_ptr`; and
- `typename std::shared_ptr<T>::element_type *` for `std::shared_ptr`.

(In other words: the return types for the respective `get()` functions.)

§ 5.1.3. Custom pointer types for `unique_ptr`

`std::unique_ptr` can use a custom pointer type through its deleter. This pointer type is only required to model `Cpp17NullablePointer`, which means that it's only guaranteed to be comparable against another pointer object of the same type, and not in general against raw pointers.

In earlier revisions of this proposal we were addressing this issue by having the comparison operators for `std::unique_ptr` work in terms of its `pointer` inner typedef, and not raw pointers. This however resulted in possible ambiguities: if `pointer` is a user-defined type, the user could have *already* defined a comparison operator between that type and `std::unique_ptr`. If we add a corresponding one in the Standard Library, this will risk breaking users' code. The calls to such an operator could become ambiguous, or, worse, change meaning (call the operator in the Standard Library rather than the users').

We do not feel comfortable with this breakage. While in general users are never supposed to define overloads/customization points/etc. for Standard Library or language entities, defining an operator overload that accepts a user-defined type as an input (as well as a library/language entity) can reasonably be seen as something that the user has the complete right to do.

For this reason, ****since R6 we are limiting the scope of the mixed comparison operators for `std::unique_ptr` only to language pointers****. In future, this limitation can be lifted if it is deemed useful to do so.

§ 5.1.4. Objects comparable to a smart pointer's pointer type

Consider a type which is already comparable to a pointer type `T *` (such as a user-defined smart pointer type): should it also become comparable to a Standard Library smart pointer type?

```
T *rawPtr;
my_smart_ptr<T> my_ptr;    // not in the Standard
std::unique_ptr<T> std_ptr; //      in the Standard

// Suppose that this works because `my_smart_ptr` has an overloaded operator==
if (rawPtr == my_ptr) { ~~~ }

// Then should this also work?
if (std_ptr == my_ptr) { ~~~ }
```

In principle one could conclude that the last line should also work, as the comparison semantics for smart pointers follow the identical model of comparisons for a raw pointer. However, once more, it is perfectly allowed for a user-defined type to have already defined an overload of `operator==` between itself and a Standard Library smart pointer class template; redefining the same operator again can easily create source incompatibilities.

For this reason, we are **not proposing this extension**, and we are limiting the scope of the proposed operators to comparisons between smart pointer classes and raw pointers.

§ 5.1.5. Should mixed comparisons between *different* Standard Library smart pointer classes be allowed?

In principle, we could define comparisons between different Standard Library smart pointer classes. Although [\[SD-8\]](#) isn't crystal clear on this, it is reasonable to expect that users are not allowed to define operators between two types that they do not control, so there is no danger at breaking user code here.

There are some considerations that in our opinion apply to this case.

The first is whether such an operation makes sense. From the abstract point of view of "comparing the addresses of two objects", the operation is meaningful, even if the addresses are represented by instances of different smart pointer classes. As mentioned before, the currently existing comparison operators of the smart pointer classes implement these semantics.

On the other hand: the Standard Library smart pointer classes that this proposal deals with are all *owning* classes. One could therefore reason that there is little utility at allowing a mixed comparison, because it's likely to be a mistake by the user. In a "ordinary" program, such comparisons would inevitably yield false, because the same object cannot be owned by two smart pointers of different classes (if it is, there is a bug in the program).

There is some semantic leeway here, represented by the fact that the smart pointer classes can hold custom deleters (incl. empty/no-op deleters), aliased pointers (in the case of `std::shared_ptr`), as well as fancy pointer types (in the case of `std::weak_ptr`). In principle, one can write a perfectly legal example where the same object is owned by smart pointers of different classes, using custom deleters and/or custom fancy pointer types, and then wants to compare the smart pointers (compare the addresses of the objects owned by them).

In some ways, this is hardly a realistic use case for allowing comparisons between different smart pointer classes. The danger of misuse of such comparisons, again in "ordinary" code, seems to be strictly greater than their usefulness, given the unlikelihood of valid use cases -- in the majority of ordinary usages, the comparisons would be meaningless.

We have therefore a tension between the abstract/ideal domain of the operations, and the practical usage. The problem is that **trying to solve it via the type system alone isn't possible**. For instance, type-erased deleters (in `std::weak_ptr`) make it impossible to know if, given a `std::weak_ptr<X, D>` and a `std::weak_ptr<X>` object, a comparison between them is meaningless or instead they have somehow been designed to "work together".

In R6 we are going proposing the more conservative solution: that is, **we are not going to propose mixed comparisons between different smart pointer classes**. They can always be added at a later time if so desired.

§ 5.2. Should the comparison operators yield a total order?

The current semantics of comparisons between smart pointer classes *extend* the ones defined by the language operators when used on the respective raw pointer types: smart pointer classes use `std::less` or `std::compare_three_way` and therefore always yield a strict weak ordering, even when the built-in comparison operator for pointers would not guarantee any specific ordering.

For instance:

```
std::weak_ptr<int> a(new int(123));
std::weak_ptr<int> b(new int(456));

if (a < b)           { ~~~ } // well-defined behavior
if (a.get() < b.get()) { ~~~ } // unspecified behavior [expr.rel/4.3]
```

The operations that we are proposing would similarly always be well-defined and yield a total order, by using the very same function objects.

§ 5.3. Would these operations make the `equality_comparable_with` or `three_way_comparable_with` concepts satisfied between a smart pointer and a raw pointer?

No. Adding the operations would indeed bring the Standard Library's smart pointer classes "one step closer" to satisfy those concepts, but they would still be unsatisfied because there is no `common_reference_t` between a smart pointer and a raw pointer.

Changing that is out of scope for the present proposal (and orthogonal to it).

In general, the current situation of comparison concepts and Standard Library smart pointers is "suboptimal". For instance:

```
static_assert(equality_comparable_with<unique_ptr<int>, nullptr_t>); // (1) OK since C++11
static_assert(equality_comparable_with<shared_ptr<int>, nullptr_t>); // (2) OK

static_assert(three_way_comparable_with<unique_ptr<int>, nullptr_t>); // (3) ERROR
static_assert(three_way_comparable_with<shared_ptr<int>, nullptr_t>); // (4) ERROR
```

... despite the existence of the related operators between smart pointers and `std::nullptr_t`.

- (1) and (3) used to fail in C++20 because the concepts required `unique_ptr` to be copiable.
- (3) and (4) fail because they require `std::nullptr_t` to be three-way comparable, which it isn't (`std::nullptr_t` lacks relational operations).

An analysis and discussion is available in [this thread on StackOverflow](#).

[\[P2404R3\]](#) (merged in C++23) and [\[P2405R0\]](#) (unactive) aim at closing these semantics gaps (for `std::nullptr_t`, not for raw pointers in general).

An unfortunate consequence of this is that, for the moment being, we will yet not be able to use many range-based algorithms using heterogeneous comparisons:

```
std::vector<std::unique_ptr<Object>> objects;

Object *ptr = ~~~;

auto it = std::ranges::find(objects, ptr); // ERROR; must use find_if and a predicate :-)
```

On the other hand, non-range algorithms would work just fine:

```
auto it = std::find(objects.begin(), objects.end(), ptr); // OK
erase(objects, ptr); // OK
```

Future evolutions can close this gap by providing the missing building blocks.

§ 5.4. Should `unique_ptr` have the full set of ordering operators (`<`, `<=`, `>`, `>=`), or just `<=>`?

[\[P1614R2\]](#) added support for `operator<=>` across the Standard Library. Notably, it added `operator<=>` for `unique_ptr`, leaving the other four ordering operators (`<`, `<=`, `>`, `>=`) untouched. On the other hand, when looking at `shared_ptr`, the same paper replaced these four operators with `operator<=>`.

We believe that this was done in order to preserve the semantics for the existing operators, which are defined in terms of customization points (notably, `common_type`; `unique_ptr` can work in terms of a custom "fancy" pointer type).

We are not bound by any pre-existing semantics, and the operators we are adding only act between `unique_ptr` and a raw pointer type, so we are just proposing `operator<=>` for `unique_ptr`.

§ 6. Technical Specifications

All the proposed changes are relative to [\[N4971\]](#).

§ 6.1. Feature testing macro

Add to the list in [version.syn]:

```
#define __cpp_lib_mixed_smart_pointer_comparisons YYYYMMML // also in <memory>
```

with the value specified as usual (year and month of adoption).

§ 6.2. Proposed wording

§ 6.2.1. `unique_ptr`

Modify [unique.ptr.single.general] as shown:

```
// [unique.ptr.single.mixed.cmp], mixed comparisons
template<class T2, class D2, class U>
friend constexpr bool
operator==(const unique_ptr<T2, D2>& x, const U* y) noexcept;
template<class T2, class D2, class U>
friend constexpr compare_three_way_result_t<typename unique_ptr<T2, D2>::pointer, const U*>
operator<=>(const unique_ptr<T2, D2>& x, const U* y) noexcept;

// disable copy from lvalue
unique_ptr(const unique_ptr&) = delete;
unique_ptr& operator=(const unique_ptr&) = delete;
};
```

Add a new section at the end of the [unique.ptr.single] chapter:

???.?? Mixed comparison operators [unique.ptr.single.mixed.cmp]

```
template<class T2, class D2, class U>
friend constexpr bool
operator==(const unique_ptr<T2, D2>& x, const U* y) noexcept;
```

1 Constraints:

- is_pointer_v<typename unique_ptr<U2, D2>::pointer> is true;
- equality_comparable_with<typename unique_ptr<U2, D2>::pointer, const U*> is true.

2 Returns: `x.get()` == `y`.

```
template<class T2, class D2, class U>
friend constexpr compare_three_way_result_t<typename unique_ptr<T2, D2>::pointer, const U*>
operator<=>(const unique_ptr<T2, D2>& x, const U* y) noexcept;
```

3 Constraints:

- is_pointer_v<typename unique_ptr<U2, D2>::pointer> is true;
- three_way_comparable_with<typename unique_ptr<U2, D2>::pointer, const U*> is true.

4 Returns: `compare_three_way()(x.get(), y)`.

Modify [unique_ptr.runtime.general] as shown:

```
// mixed comparisons
template<class T2, class D2, class U>
friend constexpr bool
operator==(const unique_ptr<T2, D2>& x, const U* y) noexcept;
template<class T2, class D2, class U>
friend constexpr compare_three_way_result_t<typename unique_ptr<T2, D2>::pointer, const U*>
operator<==(const unique_ptr<T2, D2>& x, const U* y) noexcept;

// disable copy from lvalue
unique_ptr(const unique_ptr&) = delete;
unique_ptr& operator=(const unique_ptr&) = delete;
};
```

6.2.2. shared_ptr

Modify [util.smartptr.shared.general] as shown:

```
template<class U>
bool owner_before(const weak_ptr<U>& b) const noexcept;

// [util.smartptr.shared.mixed.cmp], mixed comparisons
template<class T2, class U>
friend bool operator==(const shared_ptr<T2>& a, const U* b) noexcept;

template<class T2, class U>
friend compare_three_way_result_t<typename shared_ptr<T2>::element_type*, const U*>
operator<==(const shared_ptr<T2>& a, const U* b) noexcept;

};
```

Insert a new section after [util.smartptr.shared.cmp]:

??? Mixed comparison operators [util.smartptr.shared.mixed.cmp]

```
template<class T2, class U>
friend bool operator==(const shared_ptr<T2>& a, const U* b) noexcept;
```

1 Constraints: equality_comparable_with<typename shared_ptr<T2>::element_type*, const U*> is true.

2 Returns: a.get() == b.

```
template<class T2, class U>
friend compare_three_way_result_t<typename shared_ptr<T2>::element_type*, const U*>
operator<==(const shared_ptr<T2>& a, const U* b) noexcept;
```

3 Constraints: three_way_comparable_with<typename shared_ptr<T2>::element_type*, const U*> is true.

4 Returns: compare_three_way()(a.get(), b).

§ 7. Implementation experience

A working prototype of the changes proposed by this paper, done on top of GCC 13, is available in this [GCC branch](#) on GitHub.

§ 8. Acknowledgements

Credits for this idea go to Marc Mutz, who raised the question on the [LEWG reflector](#), receiving a positive feedback.

Thanks to the reviewers of early drafts of this paper on the [std-proposals](#) mailing list.

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[MarcMutzReflector]

Marc Mutz. *unique_ptr<T> @ T* relational operators / comparison*. URL: <https://lists.isocpp.org/lib-ext/2020/07/15873.php>

[Meyers]

Scott Meyers. *Effective Modern C++, Chapter 4. Smart Pointers*. URL: <https://www.oreilly.com/library/view/effective-modern-c/9781491908419/ch04.html>

[N4971]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 18 December 2023. URL: <https://wg21.link/n4971>

[P0805R2]

Marshall Clow. *Comparing Containers*. 22 June 2018. URL: <https://wg21.link/p0805r2>

[P0919R3]

Mateusz Pusz. *Heterogeneous lookup for unordered containers*. 9 November 2018. URL: <https://wg21.link/p0919r3>

[P1614R2]

Barry Revzin. *The Mothership Has Landed: Adding <=> to the Library*. 28 July 2019. URL: <https://wg21.link/p1614r2>

[P2249-GCC]

Giuseppe D'Angelo. *P2249 prototype implementation*. URL: https://github.com/dangelog/gcc/tree/P2249_mixed_smart_pointer_comparisons

[P2273R3]

Andreas Fertig. *Making std::unique_ptr constexpr*. 9 November 2021. URL: <https://wg21.link/p2273r3>

[P2404R3]

Justin Bassett. *Move-only types for equality comparable with, totally ordered with, and three way comparable with*. 8 July 2022. URL: <https://wg21.link/p2404r3>

[P2405R0]

Justin Bassett. *nullopt_t and nullptr_t should both have operator and operator==*. 15 July 2021. URL: <https://wg21.link/p2405r0>

[R.20]

Bjarne Stroustrup; Herb Sutter. *C++ Core Guidelines, R.20: Use `unique\_ptr` or `shared\_ptr` to represent ownership*. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r20-use-unique_ptr-or-shared_ptr-to-represent-ownership

[SD-8]

Bryce Lebach. *SD-8: Standard Library Compatibility*. URL: <https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility>

[SO_unique_ptr_comparable_thread]

Why is unique_ptr not equality comparable with nullptr_t in C++20?. URL: <https://stackoverflow.com/questions/66937947/why-is-unique-ptr-not-equality-comparable-with-nullptr-t-in-c20>

[Std-proposals]

P2249 discussion on the std-proposals mailing list. URL: <https://lists.isocpp.org/std-proposals/2021/01/2308.php>

[Sutter]

Herb Sutter. *Elements of Modern C++ Style*. URL: <https://herbsutter.com/elements-of-modern-c-style/>